

Research paper

A two-stage reinforcement learning-based approach for multi-entity task allocation[☆]Aicheng Gong¹, Kai Yang¹, Jiafei Lyu, Xiu Li^{*}

Shenzhen International Graduate School, Tsinghua University, Shenzhen, 518055, China

ARTICLE INFO

Keywords:

Machine learning
Reinforcement learning
Task allocation
Dynamic allocation

ABSTRACT

Task allocation is a key combinatorial optimization problem, crucial for modern applications such as multi-robot cooperation and resource scheduling. Decision makers must allocate entities to tasks reasonably across different scenarios. However, traditional methods assume static attributes and numbers of tasks and entities, often relying on dynamic programming and heuristic algorithms for solutions. In reality, task allocation resembles Markov decision processes, with dynamically changing task and entity attributes. Thus, algorithms must dynamically allocate tasks based on their states. To address this issue, we propose a two-stage task allocation algorithm based on similarity, utilizing reinforcement learning to learn allocation strategies. The proposed pre-assign strategy allows entities to preselect appropriate tasks, effectively avoiding local optima and thereby better finding the optimal allocation. We also introduce an attention mechanism and a hyperparameter network structure to adapt to the changing number and attributes of entities and tasks, enabling our network structure to generalize to new tasks. Experimental results across multiple environments demonstrate that our algorithm effectively addresses the challenges of dynamic task allocation in practical applications. Compared to heuristic algorithms like genetic algorithms, our reinforcement learning approach better solves dynamic allocation problems and achieves zero-shot generalization to new tasks with good performance. The code is available at <https://github.com/yk7333/TaskAllocation>.

1. Introduction

Task allocation is a critical combinatorial optimization problem (Yan et al., 2012). It plays a vital role in modern applications such as multi-robot collaboration, resource scheduling, and more. In warehouse logistics, mobile robot teams allocate tasks and transport goods to different locations (Tang et al., 2021). In industrial and manufacturing fields, robotic arms are assigned to various processing tasks (Johannsmeier and Haddadin, 2016). On-demand carpooling and delivery services require the dispatch of agents to meet customer needs (Hyland and Mahmassani, 2018). As decision-makers, we must allocate tasks to entities based on their location, current attributes, and task requirements to ensure timely task completion.

Various approaches have been proposed to solve the task allocation problem (Wang et al., 2023; Quinton et al., 2023; Burkard et al., 2012; Parasuraman et al., 1996; Alighanbari and How, 2005; Merlo et al., 2023). The common approach treats it as a discrete optimization problem, assigning entities to tasks. Accurate solution algorithms like

the Hungarian algorithm (Msala et al., 2023; Samiei and Sun, 2023, 2020; Munkres, 1957), branch and bound (Martin et al., 2021; Singh, 2021; Lawler and Wood, 1966), network flow algorithms (Javanmardi et al., 2023; Jamil et al., 2023; De Weerd et al., 2007), fuzzy logic algorithm (Ali and Sridevi, 2024; Sharma et al., 2023; Ali et al., 2021; Jamil et al., 2022) and dynamic programming (Choudhury et al., 2022; Alkaabneh et al., 2021; Bellman, 2010) are used, as well as heuristic algorithms such as the genetic algorithm (DENG et al., 2023; Patel et al., 2020; Ye et al., 2020; Page et al., 2010; Forrest, 1996; Fu et al., 2023b), particle swarm optimization (Kalimuthu and Thomas, 2024; Geng et al., 2021; Qingtian, 2021), symbiotic organisms search (Truong et al., 2020; Gharehchopogh et al., 2020; Abdullahi et al., 2023), simulated annealing (Barbosa et al., 2023; Wang et al., 2022; Barboza and Kniess, 2024; Bertsimas and Tsitsiklis, 1993) are used to solve this problem. There are also methods using game theory to allocate robots to finish tasks (Martin et al., 2023; Sun et al., 2023). However, while these methods have proven effective in addressing the task allocation

[☆] This work was supported in part by the Science and Technology Innovation 2030-Key Project under Grant 2021ZD0201404.

^{*} Corresponding author.

E-mail addresses: gac19@mails.tsinghua.edu.cn (A. Gong), yk22@mails.tsinghua.edu.cn (K. Yang), lvjf20@mails.tsinghua.edu.cn (J. Lyu), li.xiu@sz.tsinghua.edu.cn (X. Li).

¹ The first two authors contribute equally to this work.

problem under static conditions, they often assume static task and entity attributes. In reality, task assignments frequently encounter dynamic states and fluctuations in task estimation or entity numbers over time. As a result, the applicability of the aforementioned methods may diminish, necessitating the exploration of new approaches for dynamic task allocation. This dynamic nature underscores the importance of developing methodologies capable of adapting to evolving conditions and uncertainties in real-time task assignment scenarios.

To address the dynamic allocation problem in real-world scenarios, our aim is to develop more efficient and practical algorithms compared to prior methods. After each step, a series of tasks can emerge, requiring us to allocate entities effectively to achieve task goals. However, these resources come with associated costs, such as power consumption in transmission and additional expenses incurred by companies when assigning employees to extra tasks. Therefore, we must carefully select the appropriate resources from a large pool to minimize costs and successfully complete tasks. Treating this problem as a Markov Decision Process (MDP), we leverage Reinforcement Learning (RL) algorithms to dynamically adjust allocation actions based on the current state, without incurring additional computational costs. Several RL-based methods have been developed for task allocation in various domains. For instance, [Afrin et al. \(2023\)](#) integrates edge and cloud computing with robotics to address computation tasks impacted by uncertain failures, utilizing a multiple deep Q-network mechanism for dynamic task allocation and ensuring quicker resiliency management. Similarly, [Fu et al. \(2023a\)](#) applies deep reinforcement learning to mobile crowd sensing, using a double deep Q network based on the dueling architecture to manage dynamic task allocation under multiple constraints, surpassing traditional heuristic methods in platform profit and task completion rate. These approaches show promising results in Smart Farm and mobile crowd sensing scenarios. Additionally, methods such as [Park et al. \(2021\)](#) and [Agrawal et al. \(2023\)](#) use attention modules to extract task and robot properties, facilitating efficient allocation decisions in environments like warehouses. Other notable works include task allocation for workers in spatio-temporal crowdsourcing ([Zhao et al., 2023](#)) and mobile crowdsensing ([Xu and Song, 2023](#)).

While RL algorithms have been widely used, there is a lack of research on allocating varying numbers of agents and handling scenarios with emerging tasks. Simply adopting traditional RL algorithms is inadequate for handling dynamically changing numbers and attributes of agents in practical applications. This is because RL predefines the number of agents and establishes a fixed network structure to calculate the value of each agent's actions in each state. When the number of agents suddenly increases, the neural network cannot allocate tasks for the additional agents. The methods mentioned above, which utilize reinforcement learning, only consider problems with a fixed number of agents and aim to improve the coordination of existing robots to accomplish tasks more effectively. Additionally, these robots have fixed characteristics, and there is no need to consider issues such as which agents' resource attributes are more suitable for allocation or which agents remain stationary on standby. Hence, to address this challenge, this paper introduces a two-stage task allocation algorithm that leverages the similarity between agent and task attributes, rendering RL algorithms applicable to the task allocation problem. Compared with heuristic algorithms, our proposed reinforcement learning method can achieve better results in dynamic task allocation and easily solve the problem of variable number of entities. When the attributes or number of tasks or entities change, our method can achieve good generalization results without training, and can better handle a series of allocation problems. Our main contributions are fourfold:

1. We introduce a pre-assign method for efficient task allocation. Firstly, the agent is pre-assigned to a task as a candidate agent for task selection, and then the agent is selected based on the degree of correlation between the task and the agent.

Table 1

Table of abbreviations.

Abbreviation	Definition
RL	Reinforcement Learning
NP	Non-deterministic Polynomial
MDP	Markov Decision Process
UAV	Unmanned Aerial Vehicle
AMIX	Attention MIX network
SHN	Self-attention-based Hyper Network
SAC	Soft Actor-Critic method
QMIX	Q MIXing method
DDPG	Deep Deterministic Policy Gradient method
EPT	Electric Power Transportation environment
LBF	Level-Based Foraging environment
RBF	Resource-Based Foraging environment
MT	Material Transportation environment
DVRP	Dynamic Vehicle Routing Problem
GA	Genetic Algorithm
PSO	Particle Swarm Optimization
SOS	Symbiotic Organisms Search

2. We incorporate Actor-Critic structure in combinational optimization problems by proposing a two-head attention mechanism module. It calculates the values of Actor and Critic simultaneously based on the similarity between agent and task attributes, enabling task allocation for a variable number of agents and zero-shot generalization of new tasks.
3. An attention-based hyperparameter network structure is proposed to estimate the overall value of critical outputs for different numbers of agents, facilitating fine-tuning of the variable number of agents in new scenarios.
4. We propose a seq2seq-like structure, similar to PointNet, to select pre-assigned agents. It selects an appropriate number of agents with suitable attributes for each task.

For your convenience, please refer to [Table 1](#), which includes a comprehensive list of all the abbreviations used in the article.

2. Related work

Multi-robot task allocation: The overview of task allocation for multiple robots is how to allocate subtasks to each robot in a way that balances the entire load. Due to the Non-deterministic Polynomial (NP) difficulty of the problem, researchers decompose it into sub-problems or apply meta-heuristic techniques. Some studies have addressed the challenges in multi-robot task allocation, which have various objectives such as energy consumption, time, cost, fairness, and task completion. Genetic algorithm ([Patel et al., 2020](#); [Ye et al., 2020](#); [Page et al., 2010](#)), k-means clustering algorithm ([Muthusamy and Chandran, 2021](#); [Sheikh et al., 2023](#); [Elango et al., 2011](#)), and imitative learning ([Wang et al., 2020](#); [Yuvaraj et al., 2021](#); [Jebara, 2001](#)) are used to solve this problem. Most research has focused on specific applications, such as manufacturing ([Wang et al., 2021](#); [Morariu et al., 2020](#); [Giordani et al., 2010](#)), inspection ([Karami et al., 2020](#); [Zhou et al., 2023](#); [Liu et al., 2017](#)), warehouses ([Tsang et al., 2018](#); [Albert et al., 2023](#)), disaster rescue ([Ghassemi and Chowdhury, 2022](#); [Xu et al., 2023](#)), multiple unmanned aerial vehicle (UAV) formations ([Wu et al., 2023](#); [Zhang et al., 2020b](#)), computation allocation ([Sun and He, 2023](#)), and workshop scheduling ([Wang and Wu, 2023](#); [Zhang et al., 2022](#); [Han et al., 2022](#); [Dahl et al., 2009](#)). Currently, [Zhang et al. \(2023\)](#) uses graph-based method to batch tasks with time and precedence constraints, aiming to optimize coordination, reduce downtime, and minimize energy consumption to addresses multi-robot task allocation. [Wen and Ma \(2024\)](#) introduces a unified model for the multi-robot task allocation problem, presenting an efficient indicator-based multi-objective evolutionary algorithm with a hybrid encoding scheme and adaptive archive update mechanism. [Zhang et al. \(2024\)](#) proposes a

novel graph deep-reinforcement-learning-based approach that leverages graph sampling and cross-attention decoding to efficiently allocate tasks to robots, demonstrating high scalability and robustness through extensive experiments in various multi-robot task allocation scenarios. Yan and Di (2023) introducing a hyper-heuristic algorithm that minimizes the time cost for compulsory tasks while selectively completing functional tasks to address multi-robot task allocation. Guo et al. (2024) introducing a collaborative discrete bee colony algorithm. Agrawal et al. (2023) presents a novel reinforcement learning-based algorithm (RTAW) for multi-robot task allocation in warehouse environments, utilizing a deep multi-agent reinforcement learning method with an attention-inspired policy architecture to minimize total travel delay and improve makespan. This article focuses more on algorithmic innovation and provides a new allocation model based on RL algorithms for tasks, including multi-robot allocation.

Dynamic task allocation: There are several typical methods in the study of dynamic task allocation. The first method is to use a multi-robot task allocation strategy (Quinton et al., 2023; Schneider et al., 2017), which mimics the process of market trading. When a new task is born, all robots will quote, and ultimately the task is assigned to the robot with the lowest quote. Robots will regularly update their quotes to reassign tasks, but this auction is very time-consuming, so the action distance of robots assigned using this strategy will be set very short (Ullah and Nawi, 2023; De Ryck et al., 2022; Talebpour and Martinoli, 2018). The second method is to still use the static task allocation method to search for solutions. When the task state changes, fine-tuning methods are used to find new solutions. For example, the genetic algorithm in meta heuristic algorithms can partially cross mutate on the basis of the original solution to obtain a new solution (Patel et al., 2020; Zhang et al., 2021; Chen et al., 2018; Zhou et al., 2019). Some methods need to rerun the algorithm to calculate new optimal solutions, such as Integer programming algorithm (Chen et al., 2023; Li et al., 2017; Su et al., 2018) and search algorithm (Sanaj and Prathap, 2020; Zhang et al., 2020a; Mitiche et al., 2019). Whether it is fine-tuning or searching for the optimal solution again, it is a very time-consuming process, and when the task changes frequently, it is completely impossible to find a suitable solution. The third method is to use clustering algorithms to cluster similar entities into a formation, and entities with similar functions or attributes will be assigned to a formation, greatly reducing the allocation time (Sun et al., 2021; Sarkar et al., 2018). This approach can meet the requirements of real-time performance, but once the attributes of the entity are misvalued or malfunctions occur, the completion rate of the task will be greatly reduced (Mitiche et al., 2019). Currently, Dynamic Multi-Robot Task Allocation for capacitated robots using Satisfiability Modulo Theories solves the problem of handling dynamic streams of tasks with deadlines, ensuring soundness, completeness, and generality for various task specifications (Tuck et al., 2024). Merlo et al. (2023) introduces an ergonomic role allocation framework for human–robot collaboration, integrating task features and human state measurements to optimize task assignment and reduce the risk of work-related musculoskeletal disorders. Afrin et al. (2023) incorporates edge and cloud computing with robotics to support computation tasks affected by uncertain failures, proposing a multiple deep Q-network dynamic task allocation mechanism to ensure faster resiliency management. Fu et al. (2023a) leverages deep reinforcement learning methods for mobile crowd sensing task allocation, using a double deep Q network based on the dueling architecture to address dynamic task allocation under multiple constraints. These two approaches have shown promising results utilizing deep reinforcement learning in Smart Farm and mobile crowd sensing scenarios. Existing RL methods (Park et al., 2021; Agrawal et al., 2023) aim to solve the allocation problem by using attention modules to extract task and robot properties for decision-making, similar to our approach. They apply an attention-based approach to derive robot and task embeddings and utilize algorithms to allocate robots, often in warehouse environments. However, these methods model the problem

as an MDP and apply reinforcement learning but do not handle scenarios with a variable number of robots. Additionally, they assume fixed robot attributes and do not allow robots to autonomously propose bids. Also, all robots must be allocated, making it a fixed-number allocation problem. In contrast, our approach addresses more complex scenarios. Our entities can include robots, vehicles, suppliers who propose bids based on their attributes, or employees requesting salaries. Our problem definition allows for entity-specific attributes and bids, enabling managers to exclude entities with high bids and low attributes. Our method accommodates variable numbers of entities, dynamic bids, and can handle entity selection and task allocation even when the number of entities far exceeds the number of tasks.

3. Problem setup

In the environment, there are many tasks in an episode, and we need to act as managers to assign entities to solve these tasks in order to receive rewards. These entities can be workers who have their own minds and are self-interested or cars that contain a lot of resources. Choosing these entities to complete tasks requires a cost, such as overtime wages for employees and fuel consumption considerations for allocating vehicles. What we need to do is choosing these entities in a reasonable manner and allocating them to complete various tasks. Suppose there are m tasks and n entities in an environment. The number of entities n is much larger than m and these entities will follow the instructions if the manager pays a cost to them. For different states, the cost of entities is dynamic since the consumption of vehicles is different and the demands of the workers are changing.

MDP. Our setting builds on the standard formulation of the Markov Decision Process (MDP) (Sutton et al., 1998) where the agent observes a state $s \in S$ and takes an action $a \in A$. The transition probability function $P(s'|s, a)$ transits current state s to next state s' after taking action a , and the agent receives a reward r according to the reward function $r : A \times S \rightarrow \mathbb{R}$. The goal of the agent is to learn a policy $\pi(a|s)$ that maximizes the expected cumulative discounted returns $\mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$ where $\gamma \in [0, 1)$ is the discount factor. In this paper, the state of the manager comprises the positional information of all entities, attributes of carried resources, the positions of tasks, and the total resources required for tasks. Actions are defined as the task allocation scenario. The state transition matrix is based on the current task allocation scenario, where the environment returns the positions and resources of all entities and tasks after each entity's movement. The state of an entity includes its own resources and position, as well as the resource requirements and positions of all tasks. Actions involve moving in any direction, and for worker entities, there is an additional action for bidding. The state transition matrix is determined by the current state and action, returning the entity's resource position and the resource requirements and positions of all tasks for the next time step.

Task. Let $T = \langle T_1, T_2, \dots, T_m \rangle$ be the sequence of tasks arriving in an episode, and task T_i is defined as a tuple $\langle r_i, tr_{i,1}, tr_{i,2}, \dots, tr_{i,k} \rangle$ where r_i denotes the reward of completing task T_i and $tr_{i,j}$ is the quantity of resource j needed to complete task T_i . Ability, experience, or the number of items can all be interpreted as resources. We define C_i as the flag of completion of task T_i , i.e., $C_i = 1$ if task T_i is completed, otherwise $C_i = 0$.

Entities. In this paper, we extend the concept of reinforcement learning agents to entities, which can be considered roughly equivalent. Entities can be item entities (e.g., vehicles) or worker entities. The difference between the two lies in whether the entity can provide bids based on its own attributes. The former can be regarded as fixed-price untrained agents, while the latter employs reinforcement learning algorithms to dynamically adjust bidding strategies. In the subsequent descriptions, we will use the term “entity” instead of “agent”. Item entities have a cost (e.g., fuel consumption and time penalty), while worker entities have wage demands for task assignments. Each entity

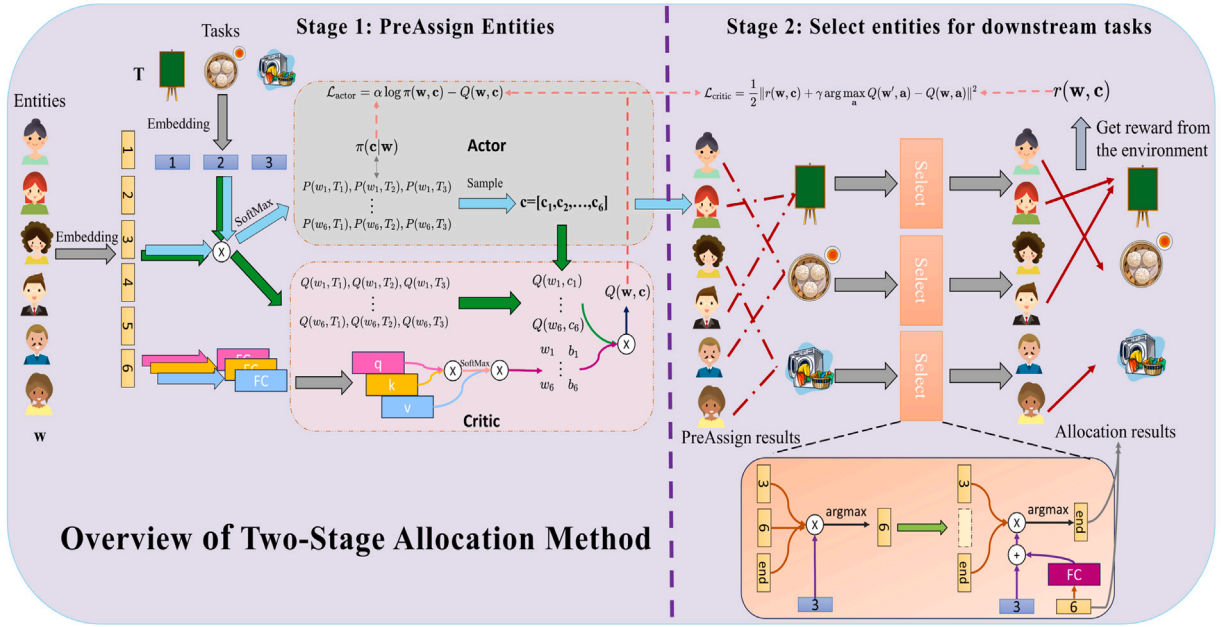


Fig. 1. The overview of our Two-Stage allocation method. First, tasks are pre-assigned based on the attributes of both tasks and entities. We utilize the PreAssign model to identify potential entities for each task, and then select the entities best suited to perform each task. Following the pre-assignment phase, each task is matched with suitable entities for its completion, resulting in the final allocation for each task.

w_i is defined as $\langle d_i, wr_{i,1}, wr_{i,2}, \dots, wr_{i,k} \rangle$, with d_i representing the cost demand and $wr_{i,j}$ denoting the resource j possessed by entity w_i . For item entities, d_i is a manually given cost function based on distance, time, purchase cost, etc. For Worker entities, they aim to maximize their individual rewards. Workers propose demands $d = \langle d_1, d_2, \dots, d_n \rangle$ based on their resources $wr = \langle wr_1, wr_2, \dots, wr_k \rangle$. A high demand reduces the chances of selection, while low demands result in lower task rewards. Worker entities take a policy $\pi_i(d_i|T)$ to determine appropriate reward demands.

Manager. Our algorithm acts as a manager, assigning entities to tasks based on task and entity attributes. The manager observes the task T , entity resources wr , entity costs d , and chooses entities ($a = \langle a_1, a_2, \dots, a_m \rangle$) to perform tasks. The manager's goal is to maximize its own reward, $\sum_{i=1}^m R_i C_i$, where $R_i = r_i - \sum_{j \in a_i} d_j$ represents the reward for task T_i with action a_i . If $R_i < 0$, the manager abandons task T_i ($C_i = 0$) due to negative profit. To maximize returns, the manager ensures selected entities have sufficient resources for task completion and minimizes entity selection costs. A policy $\pi(a|T, w)$ is learned to allocate entities to suitable tasks.

4. Methodology

The manager faces challenges in allocating entities to tasks. One challenge is the uncertainty in the number of tasks, making it difficult to allocate entities simultaneously. To address this, we propose setting the action dimension as $n \times M$, where M denotes the maximum number of tasks. However, obtaining the precise value of M in advance is challenging, and in most cases, the total number of tasks is smaller than the maximum number of tasks, which can result in wasted space. Another challenge is determining the task allocation order. Fixed order may result in suboptimal solutions, while simultaneous allocation makes it difficult to decide which task each entity should be assigned to. Additionally, the manager needs to consider the generalization problem, where entity costs and resources can vary. Most RL algorithms are unable to handle the issue of variable numbers of agents. The majority of RL algorithms typically assume a fixed environment, meaning a fixed number of agents. This assumption makes these algorithms ill-suited for handling problems with variable numbers of agents. For instance, when

the number of agents dynamically changes, traditional reinforcement learning algorithms may struggle to adapt effectively because they often require a predefined number of agents and state spaces. Therefore, when the number of agents fluctuates, these algorithms may encounter difficulties, leading to degraded performance or failure to converge.

To address these challenges, we propose a two-stage approach for entity selection. We first propose a pre-allocation method where each entity is pre-assigned as a candidate for each task based on its attributes and the attributes of the tasks. This method avoids the drawbacks of sequential and random selection by enabling sequential allocation based on the fitness between tasks and entities. We have also demonstrated that this pre-allocation method outperforms sequential allocation methods in finding optimal solutions. Secondly, our proposed attention-based hyper network structure allows for the allocation of different network parameters to different numbers of entities, enabling the neural network to accommodate dynamic entity counts and addressing the generalization issue when the number of entities changes. It overcomes local convergence and entity overlap issues and handles generalization problems. We introduce worker entities as selfish agents to verify the allocation of entities with varying attributes. The overview of our method is depicted in Fig. 1, while the architecture is illustrated in Fig. 2.

4.1. Pre-assign module

The pre-assign module initially allocates entities to suitable tasks. In discrete action space environments, each discrete number typically corresponds to an entity action. However, traditional RL algorithms focus on learning the likelihood of number occurrences in the current state, which limits their ability to effectively utilize information in task assignment problems. To leverage the individual attributes of each entity, we propose a Two-head Attention Module (TAM) based on attention mechanisms (Vaswani et al., 2017). This module guides the manager in making pre-assigned allocations based on the compatibility between tasks and entities. It provides policy and assignment value outputs, and its network structure accommodates a variable number of input entities. We adopt Soft Actor-Critic method (SAC, Haarnoja et al. (2018)) as the base RL algorithm for modeling the manager. Since

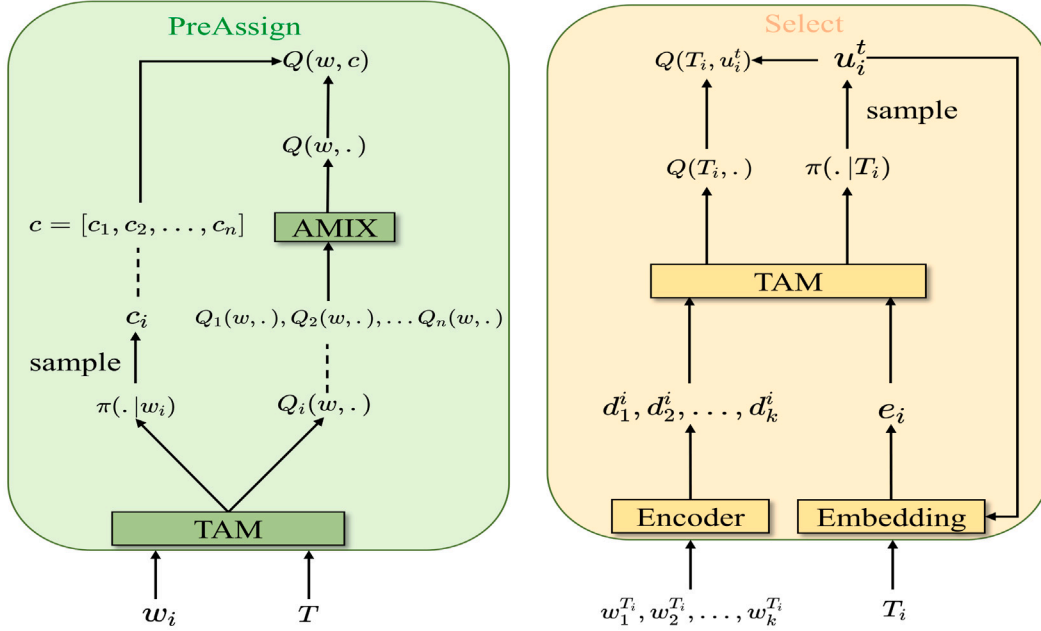


Fig. 2. The architecture of the two-stage approach. We employ the PreAssign model to preselect candidate entities for each task and subsequently choose the entities to perform each task.

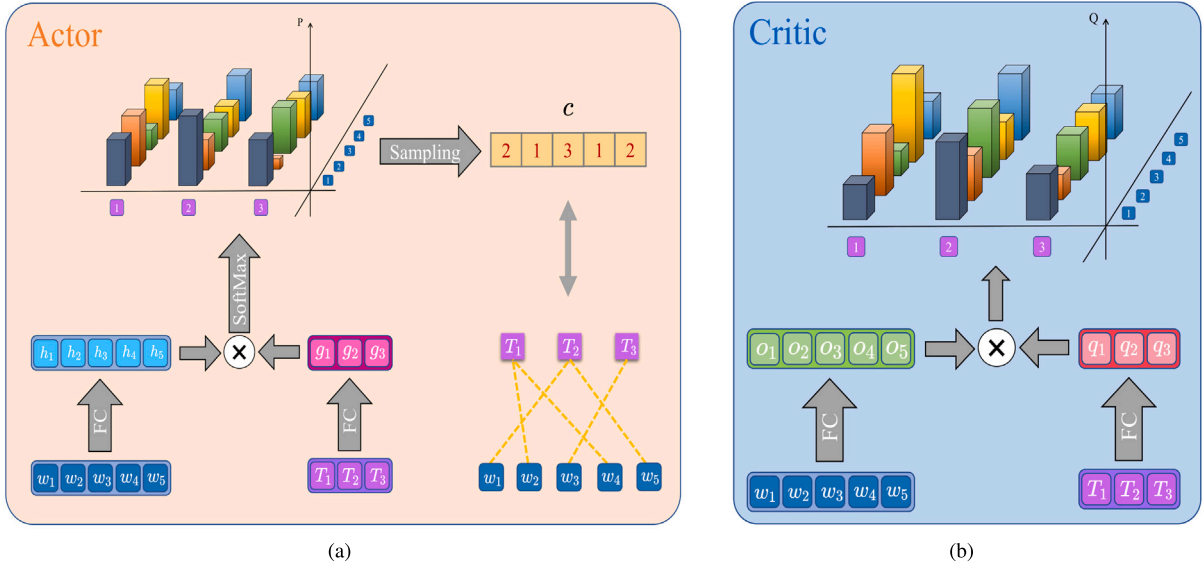


Fig. 3. The Two-head Attention Module. The actor and the critic are both computed by the two-head attention module. (a) is the actor's component, and (b) is the critic's component. The process begins with the embedding of entities and tasks. Entities are embedded as queries, while tasks are embedded as keys. The dot product of queries and keys is then computed to derive a score, which is utilized to create a distribution of pre-assigned actions in the first head and estimate the value of each action in the second head. By sampling an action from this distribution, a pre-assigned action is determined, and its value is estimated by the second head.

the action space for task allocation is discrete, we adopted the discrete version of the SAC algorithm (Christodoulou, 2019).

TAM consists of two heads: the actor head and the critic head. The actor head is responsible for generating the policy for the pre-assign actions, while the critic head calculates the value of these actions. The actor head consists of an entity embedding function $f^h : \mathbb{R}^S \rightarrow \mathbb{R}^d$, a task embedding function $f^g : \mathbb{R}^S \rightarrow \mathbb{R}^d$. Here, S represents the dimension of the task and entity attributes. The entity embedding can be expressed as $h_i = f^h(w_i)$ and the task embedding can also be denoted as $g_i = f^g(T_i)$. The probability of each entity pre-assigning to each task is computed by an attention module, i.e., $\pi(T_j|w_i) = \text{SoftMax}(h_i^T g_j / \sqrt{d})$. This method can solve the problem of a variable number of tasks with zero-shot training because when an unseen task

T_u occurs, we can compute the embedding $g_u = f^g(T_u)$ and then the probability of entity w_i assigning task T_u is $\pi(T_u|w_i) = (h_i^T g_u / \sqrt{d})$. The pre-assign allocation of entity w_i is c_i , which is sampled from the $\pi(T_j|w_i)$. The total pre-assign allocation is $c = (c_1, c_2, \dots, c_n)$ where $c_i = k$ denotes that w_i is pre-assigned to task T_k .

Due to the discrete nature of pre-assigned actions, a table can be created to record each action's value. However, the table size (2^n) becomes impractical when n is large, making it challenging to accurately approximate values with limited datasets. To address this, we adopt an approach inspired by the Q MIXing network (QMIX) (Rashid et al., 2020), which employs a mixing network to compute the total Q value for multi-agent systems. In our approach, we calculate the value selected for each entity and consider the overall assigned value

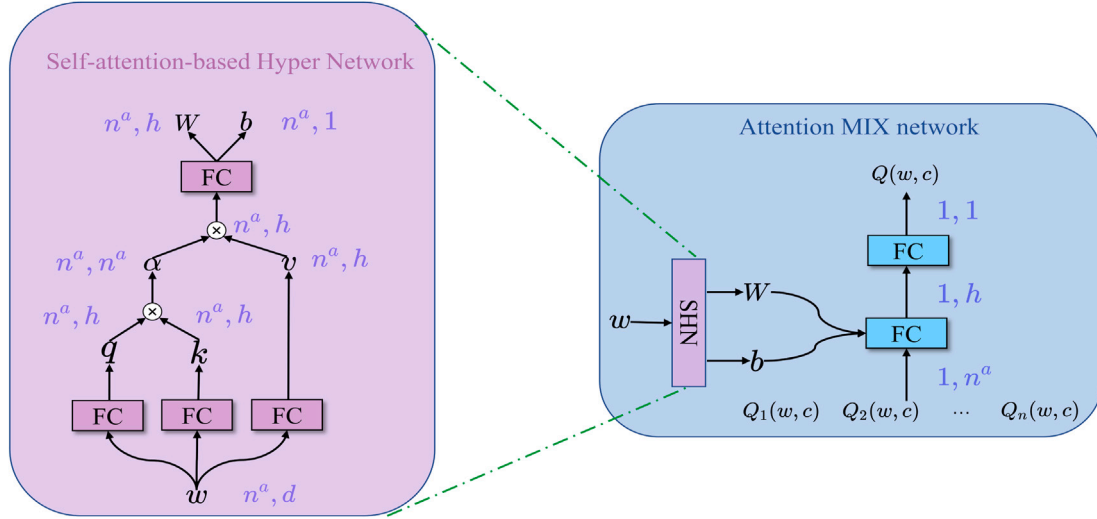


Fig. 4. The architecture of AMIX. The Self-attention-based Hyper Network (SHN) can generate variable sizes of parameters by using attention mechanisms. The input of SHN is the attributes of entities, and the outputs are the parameters of the first layer in AMIX. When there are n^a input entities, SHN will output weight parameters of dimension $n^a \times h$ and bias parameters of dimension $n^a \times 1$. Then, AMIX will take the $1 \times n^a$ dimension Q-values as input, multiply them by the weights of SHN, and add the output of the first layer network, which has a dimension of $1 \times h$. Regardless of the number of entities, the final output dimension remains the same. This allows for flexible allocation planning with varying numbers of entities. The second layer is a fully connected layer and finally outputs the estimated total value.

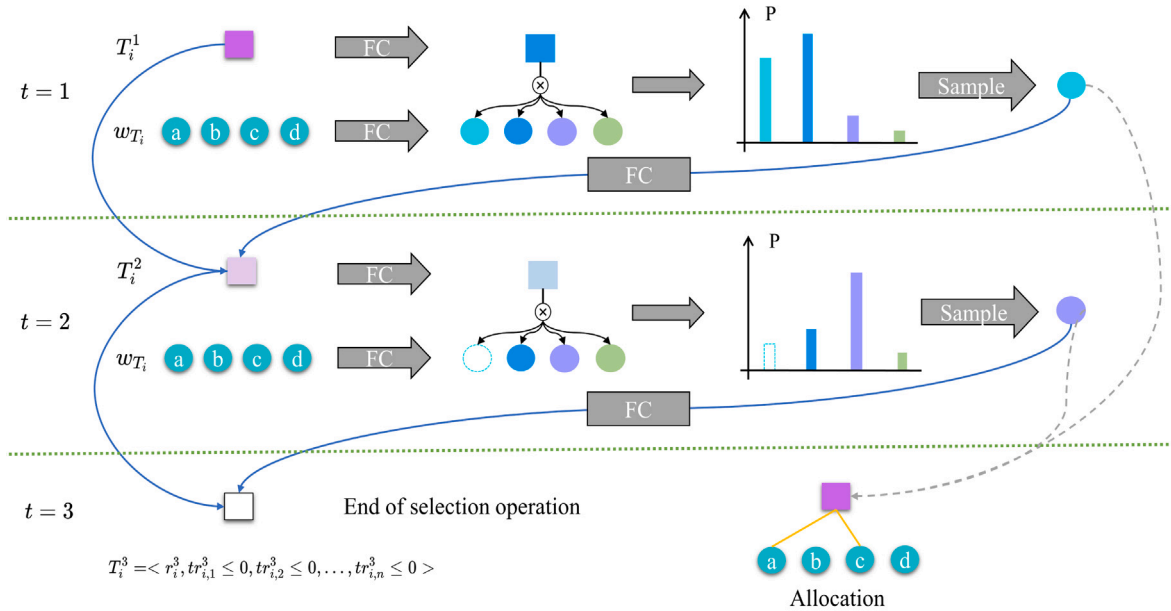


Fig. 5. The example of how select module chooses entities. The entities and tasks are embedded, and then the attention mechanism is used to compute the similarities. Similar to the method that modifies the hidden state of the sequence to indicate the word has been chosen, the entity is selected by sampling, and then it will affect the task in the following selection step to denote this entity has been chosen.

as a nonlinear aggregation of the individual entity values. In QMIX, each agent has a Q network that calculates the value of taking a particular action based on the current observations, resulting in a total of as many Q values as there are agents. Additionally, QMIX employs a hyper network that generates non-negative parameters for the neural network, combining each Q value to compute the overall Q value. For algorithm fine-tuning, we modify the attention mechanism by incorporating a critic head, similar in architecture to the actor head, to output action values. The critic component includes entity value embedding (f^o) and task value embedding (f^q) functions denoted as $o_i = f^o(w_i)$ and $q_i = f^q(T_i)$, respectively. The pre-assign value of w_j to T_i is computed using the dot product, $Q(w_i, \cdot) = o_i^T q$. The architecture of TAM is depicted in Fig. 3.

When a pre-assigned action of an entity is sampled, we obtain a value from the critic's component. In the case of having n entities, we obtain n values corresponding to the pre-assigned actions. However, since we interact with the environment throughout the entire pre-assign action, we only receive a single total reward from the environment, which does not align with the output of our critic. To address this disparity, we introduce the Attention MIXing (AMIX) module. The AMIX module utilizes a self-attention-based hypernetwork to generate parameters for its layers. Unlike QMIX, which provides network parameters for a fixed number of agents, AMIX's hyperparameter network, called the Self-attention-based Hyper Network (SHN), utilizes an attention model. This allows it to dynamically adjust the number of parameters based on the input values, effectively accommodating the

variable number of entities in the problem. For a visual representation of this concept, please refer to Fig. 4. Using the aforementioned critic structure, we can calculate the value estimates for task allocation. According to the SAC algorithm, the actor loss can be expressed as:

$$\mathcal{L}_{\text{actor}}(\theta) = \alpha \log \pi_{\theta}(\mathbf{c}|\mathbf{w}) - Q_{\phi}(\mathbf{w}, \mathbf{c})$$

After the allocation is complete, the critic is updated based on the reward values obtained from the interaction between the agent and the environment. The loss can be expressed as:

$$\mathcal{L}_{\text{critic}}(\phi) = \frac{1}{2} \left\| r(\mathbf{w}, \mathbf{c}) + \gamma \arg \max_{\mathbf{a}} Q_{\phi}(\mathbf{w}', \mathbf{a}) - Q_{\phi}(\mathbf{w}, \mathbf{a}) \right\|^2$$

Through the aforementioned loss, we can continuously update the critic network to estimate the reward values for the current allocation executed by all entities. This allows the actor network to output optimal task allocations.

4.2. Select module

When the entities are already pre-assigned to tasks by using the pre-assign module, there are many entities for each task to select. The manager needs to select the proper number of entities with different resources to complete the task. In order to have better generalization, the selection module must be able to assign entities and tasks that have not been seen before. To solve these two problems, referring to the point-net (Vinyals et al., 2015) network framework, we establish a seq2seq-like network structure as our select module. There are several reasons why the seq2seq-like structure can solve our problems. 1) The seq2seq structure is widely used for machine translation and is able to generate a variable number of outputs according to the input information. The problem of a variable number of entities can be solved. 2) In the seq2seq structure, the information of the previous output is taken into account in the context, which helps to achieve better output results in the future. This structure can also help the manager choose entities better by considering the context of previous selections. 3) The point-net attention mechanism can solve the out-of-vocabulary problem when the input and output are the same. The input and output are both entities in the allocation problem.

The architecture of the select module is shown in Fig. 5. The input of the encoder is the entities w_{T_i} pre-assigned to the task T_i , and the input of the decoder is T_i . Note that there is no sequential relationship between input entities like text, so unlike traditional seq2seq structure, we did not use the commonly used RNN and attention mechanisms to encode the input. We simply use fully connected layer as the encoder, and the output for entity w_{T_i} is d_i . The embedding of the task T_i is e_i , and the output of the i th entity u_i^t is selected by sampling from a distribution which is $\pi(\cdot|e_i^t) = \text{SoftMax}(v^T \tanh(W_1 e_i^t + W_2 d^t))$. The previous output will impact the context, so we change the task by $T_i^{t+1} = T_i^t + f^a(u_i^t)$. When the task resources needed for T_i^t are all zero, the process of selection will end.

4.3. Demand module

The worker entities have their own minds, which means they will adjust their demands in different situations. We model workers as entities that can propose their own demands based on the attributes of the task. They can update their own minds by using RL algorithms. Since each entity only considers maximizing their own reward and does not consider the global reward, and multi-agent RL algorithms focus on the optimal overall reward, which is not consistent with our scenario, we use single-agent RL algorithm to train each entity. The demand of each agent is a continuous number, so we use the DDPG (Lillicrap et al., 2015) algorithm to train these entities. The observation in DDPG is defined as the current attributes and current position of the entity, while the action is defined as the bid value for the demand. When the current entity is selected, the reward obtained is the bid value. When not selected, the reward value is 0. Therefore, worker entities need to provide reasonable bids based on their current attribute values to be selected and obtain high returns.

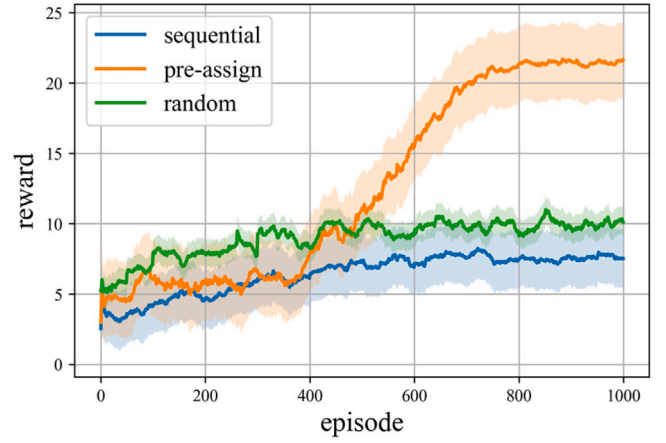


Fig. 6. The Retain the Almighty Environment. The picture shows the return of this environment by using the sequential method, the random order method, and the pre-assign method.

5. Experiment

In this section, we try to answer these questions: (1) Is the pre-assign method more effective than the sequential task allocation method in avoiding finding local optimal solutions? (2) Can the algorithm solve the allocation problem when the cost of entities is dynamically changed based on the current task? (3) Can the algorithm provide a reasonable allocation for such a variable number of entities as their number increases and decreases? (4) Does our proposed method outperform heuristic algorithms? For all experimental environments, our two-stage method uses the same hyperparameters, with no parameters specifically set for each environment. For all network architectures, we use the ReLU function as the activation function and set the number of hidden units to 64, with each network consisting of a 3-layer MLP structure. More details on the parameter configurations of our method can be found in Table 3.

5.1. Retain the almighty

This small experiment will answer the question (1) and demonstrate why using sequential selection methods can fall into local optimization and prove that using the pre-assign method is more effective.

In the Retain the Almighty environment, there are N tasks and n entities in this environment, and each task T_i in this environment corresponds to a subset of the best entities b_i , and the task is completed if and only if a best entity $w \in b_i$ participates in this task. The last task T_N is a difficult but rewarding task. The best entity of b_N is w_o , which is an almighty agent that is able to finish every task, and the cost for it is the same as others. The best allocation is to arrange $w \in b_i$ to do T_i and the task cannot choose the entities if the previous task has chosen them, which means the algorithm must retain the almighty to the last task to find a good solution. The result is shown in Fig. 6.

It can be seen that the entity learned using the sequential method is getting trapped in the local optimum. The perfect entity w_o is selected by the previous task and fails to complete the last task, resulting in a low total reward. When the order of the most difficult tasks is randomly assigned to the top, we can assign the Almighty to this task and obtain high profits; when the most difficult task is randomly assigned later, the rewards will sharply decrease. According to the training curve, the entity has not learned how to assign entities to appropriate tasks, and the final return of this entity is related to the random order of tasks instead of how well it learns to allocate entities to tasks. However, by using the pre-assign method, the manager is able to find the best

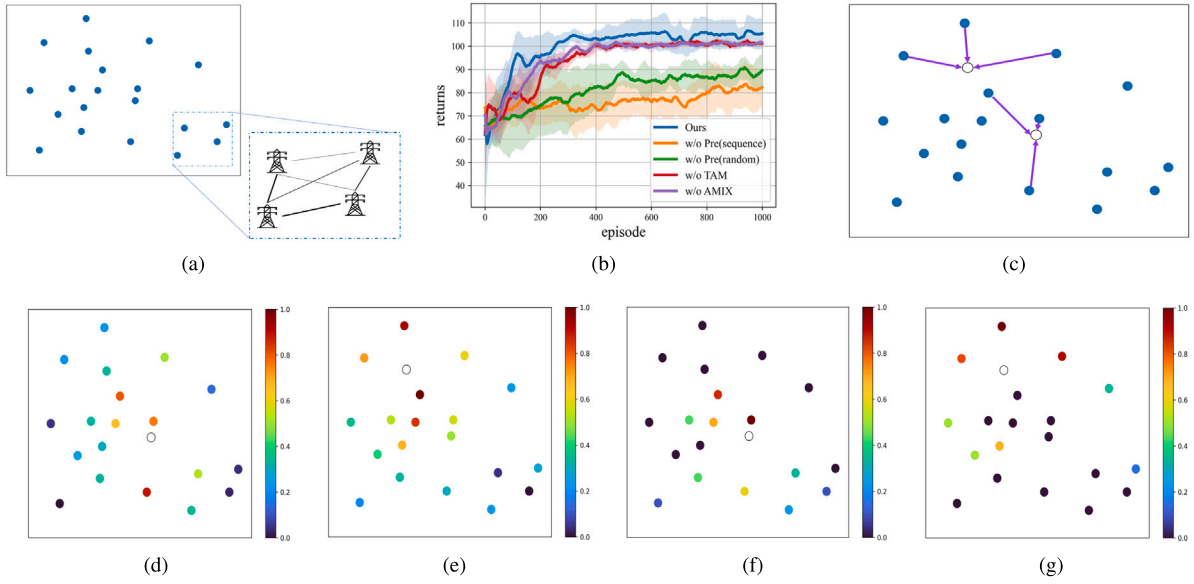


Fig. 7. The experiment with Electric Power Transportation. (a) shows the display of the environment. The blue point denotes an electric tower, and each tower is connected by wires. (b) shows the training curve with different methods. $\bigcirc$ is the task tower, and $\rightarrow$ shows the direction of power transmission. The allocation process example of our method is shown in (c)–(g). (d) and (e) show the value of the pre-assign action, and (f) and (g) show the value of selecting each entity. The value is normalized to the range of 0 and 1.

solution because the task can be completed once the best entity is pre-assigned to the last task. The manager is able to allocate w_o to T_N to get a large reward and then find the optimal solution.

In fact, in this environment, we can suppose that without prior knowledge, all entities have the same probability of being selected when the algorithm does not start training. The probability upper bound of $w_o \in T_N$ by using the sequential method is $(\frac{n-1}{n+N-2})^{N-1}$. The probability upper bound decreases exponentially with the number of tasks. However, the probability of $w_o \in T_N$ by using the pre-assign method is $\frac{1}{N}$. The rate of probability decline is far less than that of the sequential method. See Appendix A for complete proof.

5.2. Electric power transportation

In this experiment, we will answer question (2). There are many towers in the environment that are connected by wires. Different towers have different wire materials, which means different transmission costs between towers. At each time step, there may be several towers in the city corresponding to the power peak, so other towers are needed for power transmission. As a manager, we need to select the proper electric towers to carry out power transmission for the towers with peak power consumption so as to complete the task of ensuring the smooth use of electricity in the city. The towers awaiting transmission represent tasks, characterized by their locations and additional power requirements. Other towers serve as entities, characterized by their additional available power and locations. We employ our proposed two-stage method to initially allocate tasks to each tower awaiting transmission and then make selections based on the specific additional power and attributes of each tower. Once selected to participate in transportation, the cost is calculated as the distance between the two stations multiplied by the predetermined cost of transmission per meter of wire between the two towers. The transmission cost per meter of wire is determined by the material between the two towers, which is predefined in the environment. For each target tower, if sufficient power transmission is successfully obtained, the reward is the task completion reward minus the power transmission cost. Therefore, while ensuring sufficient power, reducing transmission costs is also considered. In this environment, there are a total of 20 electric towers. Each interaction with the environment causes changes in the power values required by the towers. When the power exceeds their individual limits, support from other towers is needed.

We use our two-stage approach to train the manager. To test the effect of our method, we denote *w/o Pre* as the architecture without the pre-assign module, *w/o TAM* as the architecture without the attention module, which means the actor and critic are linear layers. The *w/o AMIX* is denoted as the normal global critic to calculate the expected return of this pre-assign action. When we use the AMIX structure, we calculate the individual value assigned to each task for each entity, and then use AMIX to calculate the overall value; When the AMIX structure is not applicable, we will use an overall critical network to input the attributes of all entities and tasks and directly estimate the overall value. The training curve and an example of our allocation method are shown in Fig. 7. From the figure, it can be seen that our proposed method significantly improves the effectiveness of power allocation and increases the reward value. Additionally, the effectiveness of each proposed module is validated through the curve chart. These subplots show two towers at peak electricity usage, requiring other tower entities to transmit power. Subplot (d) illustrates the value assigned to other tower entities when pre-assigned to a central peak-usage tower. Subplot (e) shows the value for a tower located in the upper left, also at peak usage, receiving pre-assign support from other towers. After training, the pre-assign schemes with higher values are distributed around these two target towers, while towers farther away have lower values. This is due to increased transmission costs with distance, making it inefficient to pre-assign distant towers for support. Our method captures this information, enabling better pre-assignment to towers with lower transmission losses. Subplots (f) and (g) reflect the visualization results of the select stage. After pre-assignment, most tower entities are pre-assigned to the nearest peak-usage towers, and each target tower only needs to select from nearby towers. The value in the selection stage shows that target towers still prioritize nearby entities with lower transmission costs, proving that our algorithm can select suitable entities based on pre-assign results to complete the task.

5.3. Resource-based foraging (RBF)

This experiment will answer questions (2) and (3). This environment is a task assignment environment based on the benchmark environment Level-Based Foraging (LBF, Papoudakis et al., 2020) for multi-agent fully collaborative tasks and has undergone some changes to adapt to the settings in this article. The entities possess numerous

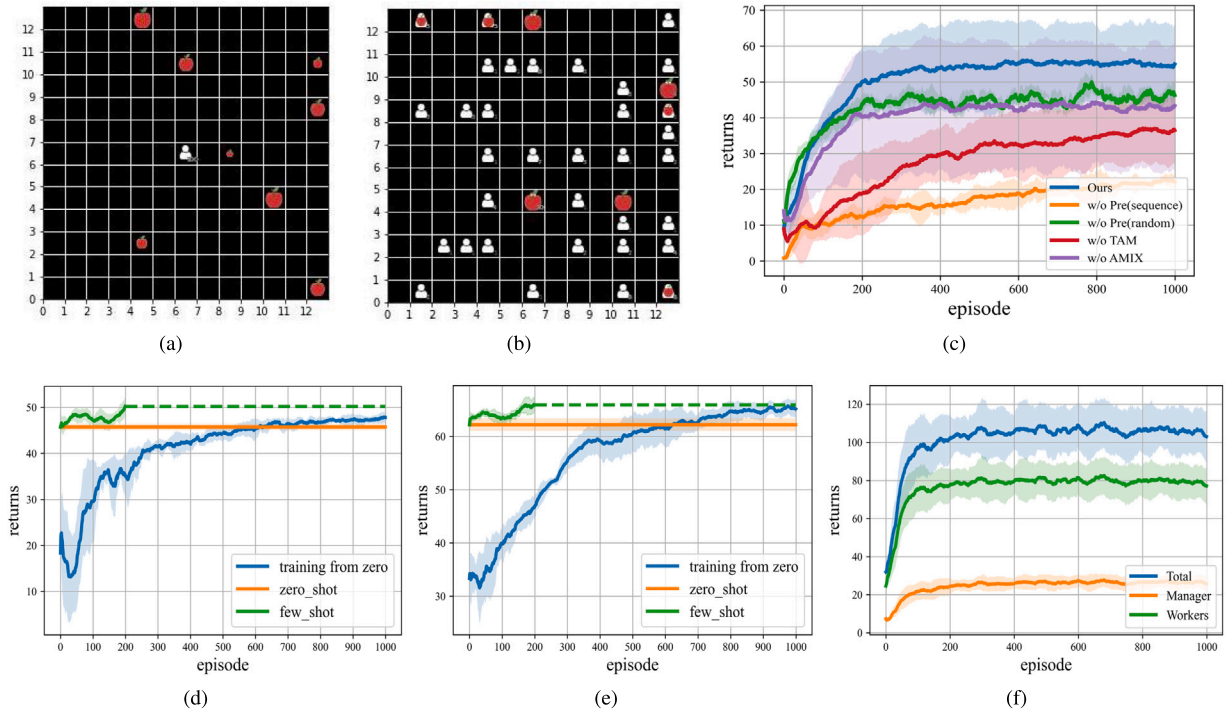


Fig. 8. The experiment of RBF. (a) shows the RBF environment. At first, all entities are in the middle of the map. With the emergence of Apple tasks, managers assign entities to complete these tasks, and (b) shows the positions of all entities at a certain moment and the current task positions. The number in the bottom right corner of the entities indicates how many entities overlap at the current position. (c) shows the changing returns of the training process. (d) and (e) show the results of the generalization experiment by changing entities and tasks. (f) is the training curve with worker entities.

resources in the RBF environment, but just one resource in the LBF environment, which represents the entities' level. For simplicity, all tasks are still represented by apples, and the differences in task resources are reflected in the size of the apples. The number of entities in RBF is very large, and selecting each entity requires a cost. Apples refresh randomly on the map over time, and the manager needs to control the entity to finish the task, i.e., pick the apples to get rewards. If the task is not completed after a certain period of time, its rewards and requirements will be reduced. Furthermore, if an entity promises to complete a task, it cannot be arranged for other tasks to be completed. In this environment, there are a total of 100 selectable entities, and every 5 time steps, 5–10 apples are randomly generated at any location on the map. The manager needs to select entities from the pool of 100 based on their attributes and positions, determining whether to choose each entity and directing them to solve specific tasks. This can be considered a large-scale task assignment problem, as there are 100 entities to allocate and a considerable number of tasks, requiring sophisticated algorithm design to address the challenge of allocating multiple tasks to a large number of entities.

We first consider that these entities are item entities whose cost is determined by the size of resources and their distance from tasks. We simplify different tasks as harvesting different apples, each with its own attributes representing the task requirements. A task is considered completed, or an apple successfully harvested, only when the total attributes of the entity at that location exceed those of the apple. Since the entity attributes are manually specified, we train only the manager in this scenario. The objective is to enable the manager to appropriately select entities based on their attributes, corresponding locations, apple positions, and the total entity attributes required for apple harvesting, and then allocate tasks accordingly. The training curve is shown in Fig. 8. To verify the generalization ability of algorithms, we first replaced the training entities with another batch of entities we had never seen before. Then we change the attributes of tasks in the environment to see the result. In the task generalization

experiment, we randomly generate new apples with different attributes on the map, distributed across various locations, and have the manager utilize the same set of entities to complete them. This task can be likened to a scenario where a company employs the same group of employees to complete historical tasks and verifies whether they can allocate the employees reasonably based on the requirements of new tasks. The results of the training process, the zero-shot generalization, the few-shot generalization, and the result of training from scratch are shown in Fig. 8. We can see that whether it is a change in the number and attributes of entities or a change in the attributes of tasks, the experimental results of zero-shot generalization of the model are almost consistent with the effect of retraining the model about 700 epochs, and the few-shot training performance of the model, which trains 200 epochs in this environment, is better than retraining it 1000 times. This demonstrates that our model does not remember how it should behave in the current environment but rather learns to assign strategies based on entity and task attributes.

We also consider the situation where the entities are worker entities, and they will demand their costs for being selected to do a task. They will dynamically adjust their quotes based on the attributes of the task, their own attributes, quotes, final returns, and the selected situation. We use the DDPG algorithm to simulate the process of entity learning to propose demand. We can see from Fig. 8 that as the manager uses our algorithms to learn how to distribute, both total and manager benefits increase. And workers gradually increase their quotes and benefits, so the rate of manager revenue increase begins to decrease. As the total income begins to reach the upper limit, the manager's income at this time reaches the upper limit, and workers no longer raise their demands, creating a dynamic balance. In the training process, workers are constantly learning and modifying the quotation, at which time the algorithm can select the appropriate workers to complete the task. This further proves that our algorithm can dynamically give a reasonable distribution result based on the current situation.

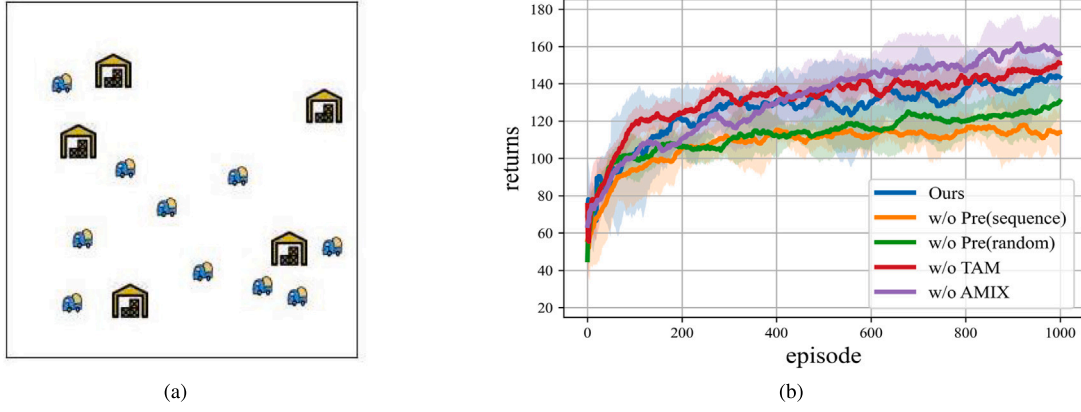


Fig. 9. The experiment of Material Transportation.

5.4. Material transportation

We use a Material Transportation environment and the problem to be solved is the Dynamic Vehicle Routing Problem (DVRP) problem. There are trucks carrying resources at various parking points on the map, and we need to allocate these trucks reasonably based on the amount of resources required by each city. We have a total of 50 entities equipped with various materials. At each time step, the environment generates material demands at random positions. The manager needs to dynamically allocate tasks to the agents based on the entities' positions and remaining material attributes. This environment is different from the previous environments since once an entity completes a task in this environment, the resources that it carries will be cleared, meaning that it cannot be selected to complete other tasks anymore. The results of this experiment are shown in Fig. 9. From the comparison, it is evident that our algorithm does not achieve the highest performance when compared to the results obtained without the AMIX or the TAM network structure. However, as mentioned earlier, networks lacking these modules are unable to handle variable number entities allocation. Therefore, while our algorithm may not excel in this specific task, it possesses the crucial ability to generalize and adapt to different scenarios.

5.5. Results

Here, we compare our proposed reinforcement learning algorithm with three mainstream heuristic algorithms, Genetic Algorithm (Holland, 1992, GA), Particle Swarm Optimization (Kennedy and Eberhart, 1995, PSO), and Symbiotic Organism Search (Cheng and Prayogo, 2014, SOS) to address question (4) and demonstrate the effectiveness of our method in dynamic task allocation. As our tasks are dynamically provided, we periodically rerun these heuristic algorithms to adapt to changing task allocations. These algorithms, originally designed for static planning problems, are now triggered for reruns every 10 interactions with the environment, considering the current set of agents and tasks. For these heuristic algorithms, the encoding length is determined by the product of the MDP length and the action space dimension. During interaction with the environment, actions are chosen based on this encoding. In the Genetic Algorithm, the mutation rate is set to 0.05, meaning each gene in the encoding has a 5% chance of undergoing a random change. The crossover rate is set to 0.5, with each crossover operation randomly selecting a crossover point and exchanging subsequent genes between parents. The population size is fixed at 100, and the algorithm runs for a total of 1000 generations, which is same as the RL method. Since each particle represents an allocation, values are constrained between 0 and the maximum number $N + 2$. When the integer part equals k ($k \geq 1$), it indicates that the entity is assigned to task k ; when the integer part is 0 or larger than

N , it indicates that the entity is not assigned to any task. For the PSO algorithm, we initialize 100 particles, set the inertia weight to 0.5, and both c_1 and c_2 to 1.5. The total number of iterations is set to 1000. In the SOS algorithm, the number of organisms is also set to 100. The search space spans from 0 to the maximum number plus one, similar to the GA setting. The coefficients during the mutualism and commensalism phases are set as random numbers between -1 and 1 .

The training curves between the number of training steps and the final return are shown in Fig. 10. It can be observed that our proposed two-stage reinforcement learning approach outperforms heuristic algorithms in dynamic task allocation planning. The final returns of the Electric Power Transportation (EPT), Resource-Based Foraging (RBF), and Material Transportation (MT) environments are presented in Table 2. We conducted a comparative analysis to evaluate the training performance and generalization ability of our proposed method against traditional approaches in these environments. In the entity-generalization task, in the EPT environment, we did not reduce the number of entities but instead changed the attributes of the entities. In the MT environment, we slightly decreased the number of entities from 50 to 40 and also changed the amount of resources carried by each entity. In the RBF environment, we reduced the number of entities from 100 to 50. For the task-generalization scenarios, we altered the probability of occurrence for each task at each time step and replaced the requirements or resources needed for the tasks. The table clearly illustrates that our proposed structure outperforms traditional structures in dynamic task allocation problems. Our algorithm achieved superior performance on most tasks, and the generalization experiments of zero-shot and few-shot learning have demonstrated its adaptability to unseen entities and tasks, resulting in excellent allocation performance.

5.6. Analysis

From the tables and experimental curves, it is clear that our proposed method outperforms in addressing dynamic task allocation problems. From the results presented in Table 2, it is evident that our proposed method outperforms GA, PSO, and SOS algorithms across all three experimental environments, demonstrating its effectiveness in solving dynamic allocation problems. As shown in Fig. 10, our training curve starts off lower than the heuristic algorithms. However, in the mid-training phase, our method surpasses the heuristics and maintains the highest performance until the end. This may be due to the neural network's need to extract features based on the current attributes of entities and tasks, which requires observing and understanding each dimension and position. This process demands a substantial amount of data to update feature extraction parameters effectively, allowing the model to learn an optimal allocation strategy based on the state of entities and tasks. In the later stages of training, when data volume increases, the reinforcement learning method's

Table 2

Analysis of our model. Our proposed network can achieve good performance in the environment and achieve generalization of zero-shot and few-shot.

Env Name	Train/Test	Ours	GA	PSO	SOS	W/o Pre(sequence)	W/o Pre(random)	W/o AMIX	W/o TAM
EPT	train	105.4 ± 6.34	72.5 ± 10.4	78.6 ± 4.2	89.4 ± 6.5	82.4 ± 8.13	89.6 ± 6.87	101.5 ± 1.44	101.0 ± 1.24
	zero-shot(entity)	125.5 ± 0.51	\	\	\	92.8 ± 5.63	101.7 ± 10.6	126.3 ± 1.61	\
	few-shot(entity)	130.1 ± 2.47	\	\	\	116.6 ± 1.63	121.3 ± 1.64	\	\
	zero-shot(task)	75.1 ± 0.65	\	\	\	64.3 ± 0.75	68.3 ± 0.83	78.3 ± 1.03	\
	few-shot(task)	82.3 ± 2.38	\	\	\	73.4 ± 1.87	77.4 ± 2.59	\	\
RBF	train	54.9 ± 11.12	46.2 ± 3.3	50.7 ± 4.0	48.1 ± 3.4	22.2 ± 0.93	46.3 ± 2.94	43.4 ± 17.85	36.5 ± 9.55
	zero-shot(entity)	45.7 ± 0.67	\	\	\	39.6 ± 0.83	35.6 ± 0.73	27.4 ± 0.58	\
	few-shot(entity)	50.2 ± 1.12	\	\	\	46.4 ± 1.67	38.6 ± 2.43	\	\
	zero-shot(task)	62.2 ± 1.19	\	\	\	26.4 ± 0.98	55.1 ± 1.21	36.7 ± 0.85	\
	few-shot(task)	65.9 ± 1.64	\	\	\	30.3 ± 2.41	57.5 ± 1.54	\	\
MT	train	143.5 ± 13.9	103.4 ± 12.5	114.7 ± 7.2	125.7 ± 7.8	114.1 ± 6.06	130.7 ± 8.79	151.0 ± 6.17	156.1 ± 18.2
	zero-shot(entity)	138.6 ± 3.87	\	\	\	98.6 ± 2.81	110.3 ± 2.75	143.6 ± 2.54	\
	few-shot(entity)	148.0 ± 9.56	\	\	\	120.3 ± 6.12	130.3 ± 4.10	\	\
	zero-shot(task)	54.3 ± 1.49	\	\	\	44.8 ± 3.56	40.4 ± 2.87	56.3 ± 2.23	\
	few-shot(task)	60.4 ± 3.62	\	\	\	49.7 ± 2.65	53.6 ± 4.13	\	\

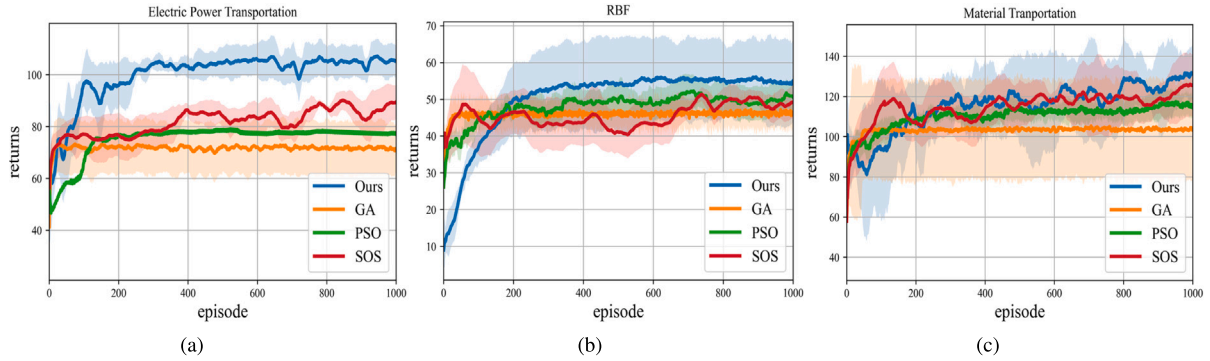


Fig. 10. The results of our method, GA, PSO, and SOS methods. We run each algorithm with 5 different seeds and displayed the mean and variance curves. It can be observed that our proposed algorithm achieves better performance compared to heuristic algorithms.

learned strategy can provide better real-time task allocation according to the scenario, resulting in a higher performance ceiling. This success can also be attributed to our modeling of the problem as an MDP, which enables real-time capture of changes in environmental tasks and entity attributes. In contrast, heuristic algorithms address static task allocation problems in each iteration and lack the adaptability of reinforcement learning algorithms in handling such dynamic scenarios. Additionally, compared to heuristic algorithms, our method can address generalization issues. When new entities or tasks appear, our algorithm can directly allocate tasks, whereas heuristic algorithms fail to handle generalization due to mismatched dimensions caused by changes in the number of entities or tasks. Furthermore, heuristic algorithms do not take into account the attributes of entities or tasks, so they cannot dynamically adjust their strategies when these attributes change.

Compared to sequential allocation or random sequential allocation, our proposed pre-allocation method achieves better results. In various environments, our method shows significant improvements, indicating that our pre-allocation approach can find better task allocation solutions. In contrast, task allocation based on natural or random order is more likely to get stuck in local optima, resulting in suboptimal allocations.

From Table 2, it is noteworthy that in some experiments, the absence of the AMIX module leads to better results in zero-shot scenarios. This can be attributed to the SHN network's utilization of an attention structure that inherently involves a large number of parameters. The SHN network's output serves as weight parameters for the TAM network. While this approach enhances fine-tuning capabilities across various tasks and scenarios, captures variations in the number of entities, and achieves superior generalization, it also increases the total number of parameters and training complexity compared to directly learning the parameters of the AMIX neural network. Consequently, zero-shot performance may slightly degrade.

In the RBF environment, employing the AMIX module results in superior performance. However, in other environments, its use leads to slightly inferior results. This variation can be attributed to the specific settings of each environment. For instance, EPT involves fewer entities and simpler tasks. In the generalization scenario, we did not modify the number of entities or total tasks but only altered the resource attributes carried by the entities and the attributes of the tasks. In the RBF environment, we employed 100 entities, whereas in zero-shot and few-shot scenarios, we reduced the number of entities and tasks by 50%. Since the attention module can better capture the impact of changes in the number of entities on task allocation, our method achieves optimal results in training and various generalization tasks within the RBF environment. It significantly outperforms the AMIX network without the attention module. Conversely, in EPT and MT scenarios, training without the attention module leads to faster convergence. Due to minimal changes in the number of entities, omitting the AMIX module yields better performance. Our proposed method performs better in task environments with large-scale and numerous entities. When there are many entities, the AMIX module can better capture the connections between entities. However, for smaller-scale tasks, the reduced number of entities makes it challenging to capture the relationships between entities and different entity counts. As a result, the proposed module does not show significant improvement during training. Nonetheless, without the AMIX module, the critic cannot accurately estimate task allocation values for a variable number of entities and tasks. It relies solely on the actor module with the attention network to output policies but cannot use the critic network to update those policies. Thus, it is not possible to fine-tune parameters for new tasks.

From the perspective of generalization, our proposed method demonstrates excellent generalization without additional training. In Fig. 8, directly applying the trained model to new environments yields results comparable to those after approximately 700 steps of retraining.

Furthermore, after fine-tuning for 100 steps, it performs better than retraining for 1000 steps, proving the zero-shot and few-shot capabilities of our method. Additionally, compared to sequential and random selection methods, our pre-allocation method also achieves better results in generalization scenarios, further validating the effectiveness and rationality of our approach.

6. Conclusion

In our work, we propose a two-stage approach to solve the task allocation problem. Our approach starts by pre-assigning entities to tasks that they are good at, based on the similarity of each entity and task. We then select from the candidate entities in each task using a sequence model similar to point-net. The proposed TAM and AMIX network architectures can accommodate the changing number of tasks and entities and have the potential to achieve zero-shot and few-shot generalization to new tasks and unseen entities scenarios. Through a variety of experiments, we verify the effectiveness of our proposed two-stage task allocation approach and the validity of our proposed structure. We compared our algorithm with heuristic algorithms in multiple environments and found that our algorithm achieves better results, demonstrating the effectiveness of our approach. Additionally, we conducted generalization experiments by modifying the number of entities and tasks, as well as the associated resources and attributes. This validation confirms that our method can successfully address these challenges and achieve good generalization performance.

7. Limitation

A limitation of our method pertains to the prerequisite knowledge of task and entity attributes for efficient task allocation, which is highly justified in resource scheduling and delivery scenarios. However, in certain contexts, it becomes imperative to make reasonably estimated attributions for tasks and entities. For instance, within an enterprise, quantifying project complexity and required competencies, while conducting quantitative evaluations of employees, becomes indispensable to leverage our algorithm for task allocation. Encouragingly, this limitation can be overcome as certain task characteristics and entity observations are readily obtainable. By employing appropriate methods to extract task and entity features and employing them as attributes, these challenges can be effectively addressed.

CRedit authorship contribution statement

Aicheng Gong: Project administration. **Kai Yang:** Writing – original draft, Methodology. **Jiafei Lyu:** Writing – review & editing, Validation. **Xiu Li:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix A. Proof

The following is a proof of the probability upper bound of the sequential selection method. We define $w \in b_{a_i}$ as the set of entities which only belong to b_i , i.e., $w \in b_{a_i} \Leftrightarrow \forall i, j \neq i, w \in b_i, w \notin b_j$. We denote $r_i = b_i \setminus b_{a_i}$. The number of entities in b_{a_i} and r_i is defined as n_{a_i} and n_{r_i} . The summary of n_{a_i} is less than $n-1$ and n_{r_i} is bigger than 1 since $\forall i, w_o \in r_i$. We record the number of entities in b_{a_j} who were mistakenly selected by T_i as m_i^j . The probability of $w_o \in T_N$ is

$$\begin{aligned} & \frac{n_{a_1}}{n_{a_1} + r_{a_1}} \frac{n_{a_2} - m_2^1}{n_{a_2} - m_2^1 + r_{a_2}} \frac{n_{a_3} - m_3^1 - m_3^2}{n_{a_3} - m_3^1 - m_3^2 + r_{a_3}} \cdots \frac{n_{a_{N-1}} - \sum_{k=1}^{N-2} m_{N-1}^k}{n_{a_{N-1}} - \sum_{k=1}^{N-2} m_{N-1}^k + r_{a_{N-1}}} \\ & \leq \frac{n_{a_1}}{n_{a_1} + r_{a_1}} \frac{n_{a_2}}{n_{a_2} + r_{a_2}} \frac{n_{a_3}}{n_{a_3} + r_{a_3}} \cdots \frac{n_{a_{N-1}}}{n_{a_{N-1}} + r_{a_{N-1}}} \left(\frac{a+c}{b+c} \geq \frac{a}{b} \quad 0 < a < b, c \geq 0 \right) \\ & \leq \frac{n_{a_1}}{n_{a_1} + 1} \frac{n_{a_2}}{n_{a_2} + 1} \frac{n_{a_3}}{n_{a_3} + 1} \cdots \frac{n_{a_{N-1}}}{n_{a_{N-1}} + 1} \\ & = \frac{1}{1 + \frac{1}{n_{a_1}}} \frac{1}{1 + \frac{1}{n_{a_2}}} \frac{1}{1 + \frac{1}{n_{a_3}}} \cdots \frac{1}{1 + \frac{1}{n_{a_{N-1}}}} \\ & = e^{-[\ln(1 + \frac{1}{n_{a_1}}) + \ln(1 + \frac{1}{n_{a_2}}) + \cdots + \ln(1 + \frac{1}{n_{a_{N-1}}})]} \end{aligned}$$

We consider the function $f(x) = \ln(1 + \frac{1}{x})$. The second derivative of the function is $\frac{2x+1}{x^2(1+x)^2}$ which is positive when $x > 0$. So the function $f(x) = \ln(1 + \frac{1}{x})$ is a convex function. By using the Jensen inequality of the convex function, we have

$$\frac{\ln(1 + \frac{1}{n_{a_1}}) + \ln(1 + \frac{1}{n_{a_2}}) + \cdots + \ln(1 + \frac{1}{n_{a_{N-1}}})}{N-1} > \ln(1 + \frac{N-1}{\sum_{i=1}^{N-1} n_{a_i}}).$$

Using the above inequality, we have

$$\begin{aligned} & e^{-[\ln(1 + \frac{1}{n_{a_1}}) + \ln(1 + \frac{1}{n_{a_2}}) + \cdots + \ln(1 + \frac{1}{n_{a_{N-1}}})]} \\ & \leq e^{-(N-1)\ln(1 + \frac{N-1}{\sum_{i=1}^{N-1} n_{a_i}})} \\ & \leq e^{-(N-1)\ln(1 + \frac{N-1}{n-1})} \\ & = \left(\frac{n-1}{N+n-2} \right)^{N-1}. \end{aligned}$$

The probability of $w_o \in T_N$ using pre-assign method is $\frac{1}{N}$ because we only need to make sure the pre-assign is correct.

Appendix B. Algorithms

In this section, we give the pseudocode of the algorithms. Algorithm 1 is the main algorithm of our approach. It shows how we assign tasks based on given tasks and entities. Algorithm 2 corresponds to the pre-assign module mentioned in the main text, which is used to pre-assign entities as the candidates for all tasks. Algorithm 3 is the select module used to select and allocate entities for each task. We used Python and PyTorch framework to implement pseudo code.

Algorithm 1 Two-stage Algorithm

Input: Tasks $T = \langle T_1, T_2, \dots, T_m \rangle$, entities $w = \langle w_1, w_2, \dots, w_n \rangle$

Output: Allocation a

- 1: Initialize all network parameters
- 2: Get pre-assign policy function and value function $\pi(\cdot|w)$, $Q(w, \cdot) = \text{Pre-Assign}(T, w)$
- 3: Sample pre-assign action $c \sim \pi(\cdot|w)$ and generate pre-selected entities for each task $w_T = w_{T_1}, w_{T_2}, \dots, w_{T_n}$ according to c
- 4: Get task allocation $a = \text{Select}(T, w_T)$
- 5: **return** a

Algorithm 2 Pre-Assign

Input: Tasks $T = \langle T_1, T_2, \dots, T_m \rangle$, entities $w = \langle w_1, w_2, \dots, w_n \rangle$
Output: Pre-assign policy function $\pi(\cdot|w)$, value function $Q(w, \cdot)$

- 1: Initialize all network parameters
- 2: Compute entity's policy embedding $h_i \leftarrow f^h(w_i)$, value embedding $o_i \leftarrow f^o(w_i)$, task policy embedding $g \leftarrow f^g(T)$ and task value embedding $q \leftarrow f^q(T_i)$.
- 3: Compute policy $\pi(\cdot|w) \leftarrow \text{SoftMax}(h_i^T g / \sqrt{d})$, allocation value $Q_i(w, \cdot) \leftarrow o_i^T q / \sqrt{d}$
- 4: Generate AMIX network parameters $W, b = \text{SHN}(w)$
- 5: Compute total value estimation $Q(w, \cdot) = \text{AMIX}(Q_1(w, \cdot), Q_2(w, \cdot), \dots, Q_n(w, \cdot))$
- 6: **return** $\pi(\cdot|w), Q(w, \cdot)$

Algorithm 3 Select

Input: Tasks T , entities w_T
Output: Allocation a

- 1: Initialize all network parameters, $t \leftarrow 0$
- 2: **for** $i = 1 : m$ **do**
- 3: $T_i^t \leftarrow T_i$
- 4: **while** $T_i^t[1 :] > 0$ **do**
- 5: Compute encoder embedding $d_i \leftarrow f^d(w_{T_i})$, task embedding $e_i^t \leftarrow T_i^t$
- 6: Compute value $v \leftarrow f^v(T_i, w_{T_i})$
- 7: Compute policy function $\pi(\cdot|e_i^t) \leftarrow \text{SoftMax}(v^T \tanh(W_1 e_i^t + W_2 d_i))$
- 8: Sample allocation $u_i^t \sim \pi(\cdot|e_i^t)$
- 9: Update task $T_i^{t+1} = T_i^t + f^a(u_i^t)$
- 10: **end while**
- 11: $a_i \leftarrow (u_i^0, u_i^1, \dots)$
- 12: **end for**
- 13: $a \leftarrow (a_1, a_2, \dots, a_m)$
- 14: **return** a

Table 3

Hyperparameters.

Name	Description	Value
lr_actor	Learning rate of all actor network	1e-4
lr_critic	Learning rate of all critic network	1e-4
lr_SHR	Learning rate of SHN network	1e-4
lr_AMIX	Learning rate of AMIX network	1e-4
optimizer	Type of optimizer	Adam
α	Coefficient of entropy loss	0.05
τ	Soft update rate	0.005
bs	Batch size	32
f	Activation function	ReLU
$ D $	Maximum size of buffer	1000
γ	Discount return	0.98
start ϵ	Initial exploration coefficient(manager)	1
min ϵ	Minimum exploration coefficient(manager)	0.05
ϵ decay rate	The decay rate of ϵ when the time step increases	0.9999
start σ	Initial exploration coefficient(worker)	0.5
min σ	Minimum exploration coefficient(worker)	0.05
σ decay rate	Decay rate of σ when the time step increases	0.9999
h	Dimensions of all linear hidden layer	64
I	Maximum iteration number	1000

Appendix C. Experiment details

Our experiments were performed by using the following hardware and software:

- GPUs: NVIDIA GeForce RTX 3090
- Python 3.10.8
- Numpy 1.23.4

- Pytorch 1.13.0

The hyperparameters of our approach are shown in Table 3.

References

- Abdullahi, M., Ngadi, M.A., Dishing, S.I., Abdulhamid, S.M., 2023. An adaptive symbiotic organisms search for constrained task scheduling in cloud computing. *J. Ambient Intell. Humaniz. Comput.* 14 (7), 8839–8850.
- Afrin, M., Jin, J., Rahman, A., Li, S., Tian, Y.C., Li, Y., 2023. Dynamic task allocation for robotic edge system resilience using deep reinforcement learning. *IEEE Trans. Syst. Man Cybern. Syst.*
- Agrawal, A., Bedi, A.S., Manocha, D., 2023. Rtaw: An attention inspired reinforcement learning method for multi-robot task allocation in warehouse environments. In: 2023 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 1393–1399.
- Albert, P.W., Rönnqvist, M., Lehoux, N., 2023. Trends and new practical applications for warehouse allocation and layout design: a literature review. *SN Appl. Sci.* 5 (12), 378.
- Ali, H.S., Rout, R.R., Parimi, P., Das, S.K., 2021. Real-time task scheduling in fog-cloud computing framework for iot applications: A fuzzy logic based approach. In: 2021 International Conference on Communication Systems & NETWORKS. COMSNETS, IEEE, pp. 556–564.
- Ali, H.S., Sridevi, R., 2024. Mobility and security aware real-time task scheduling in fog-cloud computing for IoT devices: a fuzzy-logic approach. *Comput. J.* 67 (2), 782–805.
- Alighanbari, M., How, J.P., 2005. Decentralized task assignment for unmanned aerial vehicles. In: Proceedings of the 44th IEEE Conference on Decision and Control. IEEE, pp. 5668–5673.
- Alkaabneh, F., Diabat, A., Gao, H.O., 2021. A unified framework for efficient, effective, and fair resource allocation by food banks using an approximate dynamic programming approach. *Omega* 100, 102300.
- Barbosa, V.L., Knies, J., Parpinelli, R.S., 2023. Optimization of task allocation in edge computing to industrial internet with simulated annealing. In: Anais do XX Encontro Nacional de Inteligência Artificial e Computacional. SBC, pp. 1171–1185.
- Barboza, V.G.R.L., Knies, J., 2024. Task allocation based on simulated annealing for edge industrial internet. In: International Conference on Advanced Information Networking and Applications. Springer, pp. 210–221.
- Bellman, R.E., 2010. *Dynamic Programming*. Princeton University Press.
- Bertsimas, D., Tsitsiklis, J., 1993. Simulated annealing. *Stat. Sci.* 8 (1), 10–15.
- Burkard, R., Dell'Amico, M., Martello, S., 2012. *Assignment Problems: Revised Reprint*. SIAM.
- Chen, J., Han, P., Liu, Y., Du, X., 2023. Scheduling independent tasks in cloud environment based on modified differential evolution. *Concurr. Comput.: Pract. Exper.* 35 (13), e6256.
- Chen, X., Zhang, P., Li, F., Du, G., 2018. A cluster first strategy for distributed multi-robot task allocation problem with time constraints. In: 2018 WRC Symposium on Advanced Robotics and Automation. WRC SARA, IEEE, pp. 102–107.
- Cheng, M.Y., Prayogo, D., 2014. Symbiotic organisms search: a new metaheuristic optimization algorithm. *Comput. Struct.* 139, 98–112.
- Choudhury, S., Gupta, J.K., Kochenderfer, M.J., Sadigh, D., Bohg, J., 2022. Dynamic multi-robot task allocation under uncertainty and temporal constraints. *Auton. Robots* 46 (1), 231–247.
- Christodoulou, P., 2019. Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*.
- Dahl, T.S., Mataric, M., Sukhatme, G.S., 2009. Multi-robot task allocation through vacancy chain scheduling. *Robot. Auton. Syst.* 57 (6–7), 674–687.
- De Ryck, M., Pissort, D., Holvoet, T., Demeester, E., 2022. Decentral task allocation for industrial AGV-systems with routing constraints. *J. Manuf. Syst.* 62, 135–144.
- De Weerd, M., Zhang, Y., Klos, T., 2007. Distributed task allocation in social networks. In: Proceedings of the 6th International Joint Conference on Autonomous Agents and Multiagent Systems. pp. 1–8.
- DENG, F., HUANG, H., TAN, C., FU, L., ZHANG, J., LAM, T., 2023. Multi-robot task allocation algorithm combining genetic algorithm and rolling scheduling. *J. Comput. Appl.* 43 (12), 3833.
- Elango, M., Nachiappan, S., Tiwari, M.K., 2011. Balancing task allocation in multi-robot systems using K-means clustering and auction based mechanisms. *Expert Syst. Appl.* 38 (6), 6486–6491.
- Forrest, S., 1996. Genetic algorithms. *ACM Comput. Surv. (CSUR)* 28 (1), 77–80.
- Fu, Y., Shen, Y., Tang, L., 2023a. A dynamic task allocation framework in mobile crowd sensing with D3QN. *Sensors* 23 (13), 6088.
- Fu, X., Sun, Y., Wang, H., Li, H., 2023b. Task scheduling of cloud computing based on hybrid particle swarm algorithm and genetic algorithm. *Cluster Comput.* 26 (5), 2479–2488.
- Geng, N., Chen, Z., Nguyen, Q.A., Gong, D., 2021. Particle swarm optimization algorithm for the optimization of rescue task allocation with uncertain time constraints. *Complex Intell. Syst.* 7, 873–890.

- Gharehchopogh, F.S., Shayanfar, H., Gholizadeh, H., 2020. A comprehensive survey on symbiotic organisms search algorithms. *Artif. Intell. Rev.* 53 (3), 2265–2312.
- Ghassemi, P., Chowdhury, S., 2022. Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints. *Robot. Auton. Syst.* 147, 103905.
- Giordani, S., Lujak, M., Martinelli, F., 2010. A distributed algorithm for the multi-robot task allocation problem. In: *Trends in Applied Intelligent Systems: 23rd International Conference on Industrial Engineering and Other Applications of Applied Intelligent Systems. IEA/AIE 2010, Cordoba, Spain, June 1–4, 2010, Proceedings, Part I 23*, Springer, pp. 721–730.
- Guo, H., Miao, Z., Ji, J., Pan, Q., 2024. An effective collaboration evolutionary algorithm for multi-robot task allocation and scheduling in a smart farm. *Knowl.-Based Syst.* 111474.
- Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *International Conference on Machine Learning. PMLR*, pp. 1861–1870.
- Han, W., Xu, J., Sun, Z., Liu, B., Zhang, K., Zhang, Z., Mei, X., 2022. Digital twin-based automated guided vehicle scheduling: a solution for its charging problems. *Appl. Sci.* 12 (7), 3354.
- Holland, J.H., 1992. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press.
- Hyland, M., Mahmassani, H.S., 2018. Dynamic autonomous vehicle fleet operations: Optimization-based strategies to assign AVs to immediate traveler demand requests. *Transp. Res. C* 92, 278–297.
- Jamil, B., Ijaz, H., Shojafar, M., Munir, K., 2023. IRATS: A DRL-based intelligent priority and deadline-aware online resource allocation and task scheduling algorithm in a vehicular fog network. *Ad Hoc Netw.* 141, 103090.
- Jamil, B., Ijaz, H., Shojafar, M., Munir, K., Buyya, R., 2022. Resource allocation and task scheduling in fog computing and internet of everything environments: A taxonomy, review, and future directions. *ACM Comput. Surv.* 54 (11s), 1–38.
- Javanmardi, S., Shojafar, M., Mohammadi, R., Persico, V., Pescapé, A., 2023. S-FoS: A secure workflow scheduling approach for performance optimization in SDN-based IoT-Fog networks. *J. Inf. Secur. Appl.* 72, 103404.
- Jebara, T., 2001. *Discriminative, Generative and Imitative Learning* (Ph.D. thesis). Media laboratory, MIT.
- Johannsmeyer, L., Haddadin, S., 2016. A hierarchical human-robot interaction-planning framework for task allocation in collaborative industrial assembly processes. *IEEE Robot. Autom. Lett.* 2 (1), 41–48.
- Kalimuthu, R., Thomas, B., 2024. Design of a multi-constraint PSO for resource allocation and task scheduling. *Int. J. Intell. Syst. Appl. Eng.* 12 (7s), 426–440.
- Karami, H., Darvish, K., Mastrogianni, F., 2020. A task allocation approach for human-robot collaboration in product defects inspection scenarios. In: *2020 29th IEEE International Conference on Robot and Human Interactive Communication. RO-MAN, IEEE*, pp. 1127–1134.
- Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: *Proceedings of ICNN'95-International Conference on Neural Networks, Vol. 4. IEEE*, pp. 1942–1948.
- Lawler, E.L., Wood, D.E., 1966. Branch-and-bound methods: A survey. *Oper. Res.* 14 (4), 699–719.
- Li, Z., Li, X., et al., 2017. Research on model and algorithm of task allocation and path planning for multi-robot. *Open J. Appl. Sci.* 7 (10), 511.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*.
- Liu, S., Kurniawan, E., Tan, P.H., Zhang, P., Sun, S., Ye, S., 2017. Dynamic scheduling for heterogeneous resources with time windows and precedence relation. In: *TENCON 2017-2017 IEEE Region 10 Conference. IEEE*, pp. 3045–3050.
- Martin, J.G., Frejo, J.R.D., García, R.A., Camacho, E.F., 2021. Multi-robot task allocation problem with multiple nonlinear criteria using branch and bound and genetic algorithms. *Intell. Serv. Robot.* 14 (5), 707–727.
- Martin, J.G., Muros, F.J., Maestre, J.M., Camacho, E.F., 2023. Multi-robot task allocation clustering based on game theory. *Robot. Auton. Syst.* 161, 104314.
- Merlo, E., Lamon, E., Fusaro, F., Lorenzini, M., Carfi, A., Mastrogianni, F., Ajoudani, A., 2023. An ergonomic role allocation framework for dynamic human-robot collaborative tasks. *J. Manuf. Syst.* 67, 111–121.
- Mitiche, H., Boughaci, D., Gini, M., 2019. Iterated local search for time-extended multi-robot task allocation with spatio-temporal and capacity constraints. *J. Intell. Syst.* 28 (2), 347–360.
- Morariu, C., Morariu, O., Răileanu, S., Borangiu, T., 2020. Machine learning for predictive scheduling and resource allocation in large scale manufacturing systems. *Comput. Ind.* 120, 103244.
- Msala, Y., Hamed, O., Talea, M., Aboulfatah, M., 2023. A new method for improving the fairness of multi-robot task allocation by balancing the distribution of tasks. *J. Robotics Control (JRC)* 4 (6), 743–753.
- Munkres, J., 1957. Algorithms for the assignment and transportation problems. *J. Soc. Ind. Appl. Math.* 5 (1), 32–38.
- Muthusamy, G., Chandran, S.R., 2021. Cluster-based task scheduling using K-means clustering for load balancing in cloud datacenters. *J. Internet Technol.* 22 (1), 121–130.
- Page, A.J., Keane, T.M., Naughton, T.J., 2010. Multi-heuristic dynamic task allocation using genetic algorithms in a heterogeneous distributed system. *J. Parallel Distrib. Comput.* 70 (7), 758–766.
- Papoudakis, G., Christianos, F., Schäfer, L., Albrecht, S.V., 2020. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *arXiv preprint arXiv:2006.07869*.
- Parasuraman, R., Mouloua, M., Molloy, R., 1996. Effects of adaptive task allocation on monitoring of automated systems. *Hum. Factors* 38 (4), 665–679.
- Park, B., Kang, C., Choi, J., 2021. Cooperative multi-robot task allocation with reinforcement learning. *Appl. Sci.* 12 (1), 272.
- Patel, R., Rudnick-Cohen, E., Azarm, S., Otte, M., Xu, H., Herrmann, J.W., 2020. Decentralized task allocation in multi-agent systems using a decentralized genetic algorithm. In: *2020 IEEE International Conference on Robotics and Automation. ICRA, IEEE*, pp. 3770–3776.
- Qingtian, H., 2021. An application of improved PSO algorithm in cooperative search task allocation. In: *2021 IEEE International Conference on Power Electronics, Computer Applications. ICPECA, IEEE*, pp. 580–583.
- Quinton, F., Grand, C., Lesire, C., 2023. Market approaches to the multi-robot task allocation problem: a survey. *J. Intell. Robot. Syst.* 107 (2), 29.
- Rashid, T., Samvelyan, M., De Witt, C.S., Farquhar, G., Foerster, J., Whiteson, S., 2020. Monotonic value function factorisation for deep multi-agent reinforcement learning. *J. Mach. Learn. Res.* 21 (1), 7234–7284.
- Samiei, A., Sun, L., 2020. Distributed recursive hungarian-based approaches to fast task allocation for unmanned aircraft systems. In: *AIAA Scitech 2020 Forum*. p. 0658.
- Samiei, A., Sun, L., 2023. Distributed matching-by-clone hungarian-based algorithm for task allocation of multi-agent systems. *IEEE Trans. Robot.*
- Sanaj, M., Prathap, P.J., 2020. Nature inspired chaotic squirrel search algorithm (CSSA) for multi objective task scheduling in an IAAS cloud computing atmosphere. *Eng. Sci. Technol., Int. J.* 23 (4), 891–902.
- Sarkar, C., Paul, H.S., Pal, A., 2018. A scalable multi-robot task allocation algorithm. In: *2018 IEEE International Conference on Robotics and Automation. ICRA, IEEE*, pp. 5022–5027.
- Schneider, E., Sklar, E.I., Parsons, S., 2017. Mechanism selection for multi-robot task allocation. In: *Towards Autonomous Robotic Systems: 18th Annual Conference, TAROS 2017, Guildford, UK, July 19–21, 2017, Proceedings 18*. Springer, pp. 421–435.
- Sharma, D.K., Mishra, J., Singh, A., Govil, R., Singh, K.K., Singh, A., 2023. Optimized resource allocation in IoT using fuzzy logic and bio-inspired algorithms. *Wirel. Pers. Commun.* 131 (2), 1393–1413.
- Sheikh, M.S., Enam, R.N., Qureshi, R.I., 2023. Machine learning-driven task scheduling with dynamic K-means based clustering algorithm using fuzzy logic in FOG environment. *Front. Comput. Sci.* 5, 1293209.
- Singh, P., 2021. Scheduling tasks based on branch and bound algorithm in cloud computing environment. In: *2021 8th International Conference on Signal Processing and Integrated Networks. SPIN, IEEE*, pp. 41–46.
- Su, X., Wang, Y., Jia, X., Guo, L., Ding, Z., 2018. Two innovative coalition formation models for dynamic task allocation in disaster rescues. *J. Syst. Sci. Syst. Eng.* 27, 215–230.
- Sun, G., Cong, Y., Dong, J., Liu, Y., Ding, Z., Yu, H., 2021. What and how: generalized lifelong spectral clustering via dual memory. *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (7), 3895–3908.
- Sun, Y., He, Q., 2023. Joint task offloading and resource allocation for multi-user and multi-server MEC networks: A deep reinforcement learning approach with multi-branch architecture. *Eng. Appl. Artif. Intell.* 126, 106790.
- Sun, Z., Sun, G., Liu, Y., Wang, J., Cao, D., 2023. BARGAIN-MATCH: A game theoretical approach for resource allocation and task offloading in vehicular edge computing networks. *IEEE Trans. Mob. Comput.*
- Sutton, R.S., Barto, A.G., et al., 1998. *Introduction to Reinforcement Learning*, vol. 135, MIT press Cambridge.
- Talebpoor, Z., Martinoli, A., 2018. Multi-robot coordination in dynamic environments shared with humans. In: *2018 IEEE International Conference on Robotics and Automation. ICRA, IEEE*, pp. 4593–4600.
- Tang, H., Wang, A., Xue, F., Yang, J.X., Cao, Y., 2021. A novel hierarchical soft actor-critic algorithm for multi-logistics robots task allocation. *IEEE Access* PP (99), 1.
- Truong, K.H., Nallagownden, P., Elamvazuthi, I., Vo, D.N., 2020. A quasi-oppositional-chaotic symbiotic organisms search algorithm for optimal allocation of DG in radial distribution networks. *Appl. Soft Comput.* 88, 106067.
- Tsang, K.F.E., Ni, Y., Wong, C.F.R., Shi, L., 2018. A novel warehouse multi-robot automation system with semi-complete and computationally efficient path planning and adaptive genetic task allocation algorithms. In: *2018 15th International Conference on Control, Automation, Robotics and Vision. ICARCV, IEEE*, pp. 1671–1676.
- Tuck, V.M., Chen, P.-W., Fainekos, G., Hoxha, B., Okamoto, H., Sastry, S.S., Seshia, S.A., 2024. SMT-based dynamic multi-robot task allocation. *arXiv preprint arXiv:2403.11737*.
- Ullah, A., Nawi, N.M., 2023. An improved in tasks allocation system for virtual machines in cloud computing using HBAC algorithm. *J. Ambient Intell. Humaniz. Comput.* 14 (4), 3713–3726.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 30.
- Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks. *Adv. Neural Inf. Process. Syst.* 28.
- Wang, X., Ning, Z., Guo, S., Wang, L., 2020. Imitation learning enabled task scheduling for online vehicular edge computing. *IEEE Trans. Mob. Comput.* 21 (2), 598–611.
- Wang, Y., Shi, Y., Liu, Y., 2022. Research on improved genetic simulated annealing algorithm for multi-UAV cooperative task allocation. In: *Journal of Physics: Conference Series*, vol. 2246, no. 1, IOP Publishing, 012081.
- Wang, Z., Wu, Y., 2023. An ant colony optimization-simulated annealing algorithm for solving a multiload AGVs workshop scheduling problem with limited buffer capacity. *Processes* 11 (3), 861.
- Wang, G., Zhang, G., Guo, X., Zhang, Y., 2021. Digital twin-driven service model and optimal allocation of manufacturing resources in shared manufacturing. *J. Manuf. Syst.* 59, 165–179.
- Wang, L., Zhu, D., Pang, W., Zhang, Y., 2023. A survey of underwater search for multi-target using Multi-AUV: Task allocation, path planning, and formation control. *Ocean Eng.* 278, 114393.
- Wen, C., Ma, H., 2024. An indicator-based evolutionary algorithm with adaptive archive update cycle for multi-objective multi-robot task allocation. *Neurocomputing* 127836.
- Wu, Y., Xu, S., Dai, W., Lin, L., 2023. Heuristic position allocation methods for forming multiple UAV formations. *Eng. Appl. Artif. Intell.* 118, 105654.
- Xu, Y., Li, X., Li, Q., 2023. A discrete teaching–learning based optimization algorithm with local search for rescue task allocation and scheduling. *Appl. Soft Comput.* 134, 109980.
- Xu, C., Song, W., 2023. Intelligent task allocation for mobile crowdsensing with graph attention network and deep reinforcement learning. *IEEE Trans. Netw. Sci. Eng.* 10 (2), 1032–1048.
- Yan, F., Di, K., 2023. Solving the multi-robot task allocation with functional tasks based on a hyper-heuristic algorithm. *Appl. Soft Comput.* 146, 110628.
- Yan, Z., Jouandeau, N., Ali-Chérif, A., 2012. Multi-robot heuristic goods transportation. In: *2012 6th IEEE International Conference Intelligent Systems*. IEEE, pp. 409–414.
- Ye, F., Chen, J., Tian, Y., Jiang, T., 2020. Cooperative task assignment of a heterogeneous multi-UAV system using an adaptive genetic algorithm. *Electronics* 9 (4), 687.
- Yuvaraj, N., Karthikeyan, T., Praghash, K., 2021. An improved task allocation scheme in serverless computing using gray wolf optimization (GWO) based reinforcement learning (RL) approach. *Wirel. Pers. Commun.* 117 (3), 2403–2421.
- Zhang, Q., Gui, L., Hou, F., Chen, J., Zhu, S., Tian, F., 2020a. Dynamic task offloading and resource allocation for mobile-edge computing in dense cloud RAN. *IEEE Internet Things J.* 7 (4), 3282–3299.
- Zhang, Z., Jiang, X., Yang, Z., Ma, S., Chen, J., Sun, W., 2024. Scalable multi-robot task allocation using graph deep reinforcement learning with graph normalization. *Electronics* 13 (8), 1561.
- Zhang, F., Mei, Y., Nguyen, S., Tan, K.C., Zhang, M., 2021. Multitask genetic programming-based generative hyperheuristics: A case study in dynamic scheduling. *IEEE Trans. Cybern.* 52 (10), 10515–10528.
- Zhang, T., Wang, Y., Liu, Y., Xu, W., Nallanathan, A., 2020b. Cache-enabling UAV communications: Network deployment and resource allocation. *IEEE Trans. Wireless Commun.* 19 (11), 7470–7483.
- Zhang, L., Zhao, J., Lamon, E., Wang, Y., Hong, X., 2023. Energy efficient multi-robot task allocation constrained by time window and precedence. *IEEE Trans. Autom. Sci. Eng.*
- Zhang, Y., Zhu, H., Tang, D., Zhou, T., Gui, Y., 2022. Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems. *Robot. Comput.-Integr. Manuf.* 78, 102412.
- Zhao, B., Dong, H., Wang, Y., Pan, T., 2023. A task allocation algorithm based on reinforcement learning in spatio-temporal crowdsourcing. *Appl. Intell.* 53 (11), 13452–13469.
- Zhou, Q., Qu, S., Wang, Q., She, Y., Yu, Y., Bi, J., 2023. Sliding window-based machine learning for environmental inspection resource allocation. *Environ. Sci. Technol.* 57 (44), 16743–16754.
- Zhou, X., Wang, H., Ding, B., Hu, T., Shang, S., 2019. Balanced connected task allocations for multi-robot systems: An exact flow-based integer program and an approximate tree-based genetic algorithm. *Expert Syst. Appl.* 116, 10–20.